Paso de argumentos y arreglos en C++

Por valor · por referencia · arreglos y arreglos multidimensionales

Alberto López

Programación Avanzada · Maestría en Ciencias e Ingeniería · UACBI, UAN

Lo que vamos a ver

El hilo de la sesión

- **Una sola pregunta** sostiene las cuatro horas: ¿qué le llega realmente a una función cuando la llamas?
- Tres respuestas posibles: una **copia** del dato, un **alias** del original, o una **dirección**.
- El paso por valor y el paso por referencia son las dos primeras.
- Los arreglos son la tercera, y por eso se ven en la misma sesión.
- Cada bloque termina con **código ejecutándose**, no con una diapositiva.

Programa de la sesión

Bloque	Contenido	Min
0	Apertura y repaso de funciones	10
1	Modelo de memoria y mecánica de la llamada	15
2	Paso por valor	30
3	Práctica guiada 1	15
	Receso	10
4	Paso por referencia	35
5	const, punteros y referencias	20
6	Práctica guiada 2	15
	Receso	10
7	Arreglos unidimensionales	25
8	Arreglos como parámetros: el decaimiento	20
9	Arreglos multidimensionales	25
10	Alternativas modernas y cierre	10

Vocabulario: parámetro y argumento

Lo vamos a usar toda la sesión

Término	Qué es	Dónde vive
Parámetro	la variable declarada en la lista de la función	dentro de la función
Argumento	la expresión escrita en la llamada	en el llamador

La pregunta del día: ¿qué relación hay entre el parámetro y el argumento?

Parámetro y argumento

```
void f(int parametro);          // 'parametro' es el parámetro

int main() {
    int x = 5;

    f(x);                       // 'x' es el argumento
    f(x + 1);                   // el argumento puede ser cualquier expresión
    f(42);                      // ...incluso un literal
}
```

Dos respuestas posibles: ninguna relación salvo el valor copiado, o el mismo objeto con dos nombres.

1

Modelo de memoria

15 minutos

- Dónde vive cada cosa
- El marco de pila
- Qué ocurre exactamente en una llamada

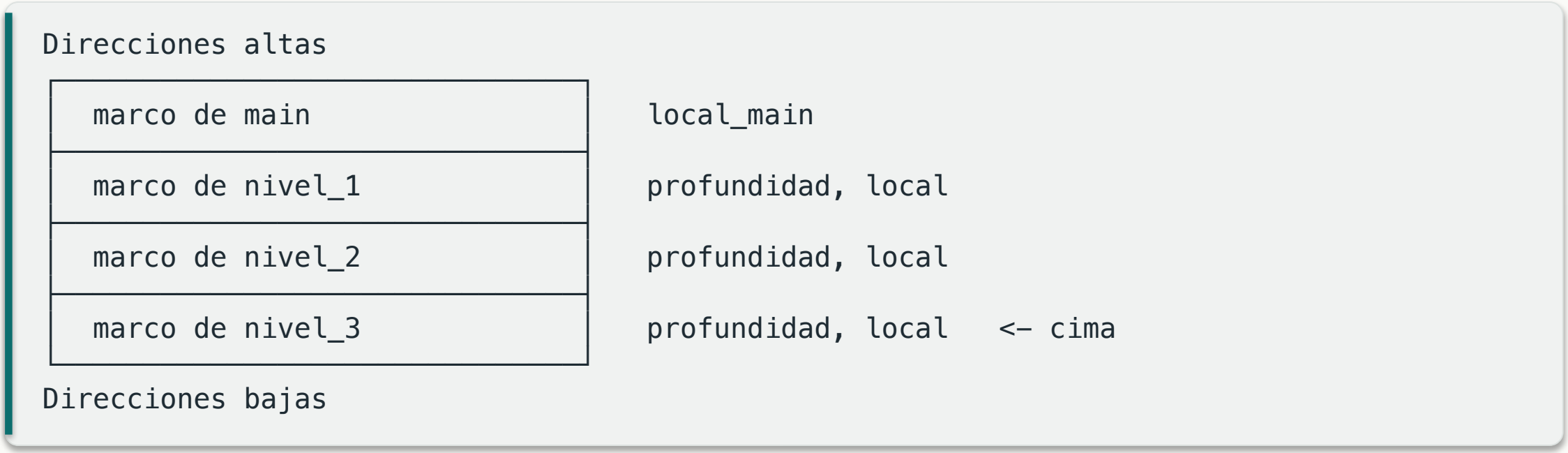
Las cuatro regiones de memoria

Región	Duración	Qué contiene	Quién la administra
Código	estática	las instrucciones compiladas	el cargador del sistema
Datos estáticos	estática	globales, `static`, literales de cadena	el compilador
Pila	**automática**	**parámetros, variables locales**	**el compilador**
Montón	dinámica	new`, y el interior de `vector` y `string	el programador o un contenedor

Todo lo declarado dentro de una función sin static tiene duración automática: nace y muere con su ámbito.

El marco de pila

Cada llamada abre uno; al volver, se descarta



En arm64 y x86-64 la pila crece hacia direcciones menores.

Qué ocurre al llamar a f(x)

void f(int p) — paso por valor

- Se evalúa la expresión `x`.
- Se reserva espacio para el marco de `f`.
- **Se inicializa el parámetro** copiando el valor del argumento.
- Se ejecuta el cuerpo de la función.
- Se destruyen los objetos locales y se libera el marco.

- Con un parámetro `int&` cambia solo el paso 3: el parámetro **no es un objeto nuevo**, se liga al objeto del llamador.

Demostración en vivo

01_marco_de_pila.cpp

```
./build/bin/exp_01_marco_de_pila
```

- → Las direcciones **bajan** conforme se anidan las llamadas.
- → Dos cadenas de llamadas consecutivas reutilizan **la misma memoria**.
- → Un parámetro es una variable local más del marco.

2

Paso de argumentos por valor

30 minutos

- Qué se copia exactamente
- El intercambio que no intercambia
- Cuánto cuesta una copia
- Cuándo es la opción correcta

En el paso por valor, el parámetro es un **objeto independiente** que se inicializa copiando el valor del argumento.

A partir de ahí, no existe ninguna relación entre los dos.

Paso por valor

```
void intenta_modificar(int copia) {
    std::cout << &copia;      // dirección A
    copia = 999;              // modifica SOLO la variable local
}

int main() {
    int original = 42;
    std::cout << &original;   // dirección B, distinta de A

    intenta_modificar(original);

    std::cout << original;    // sigue valiendo 42
}
```

Direcciones distintas significan objetos distintos. Solo se copió el valor inicial.

Demostración en vivo

02_paso_por_valor.cpp

```
./build/bin/exp_02_paso_por_valor
```

- → Preguntar **antes** de ejecutar qué va a imprimir.
- → Comparar las direcciones de dentro y de fuera.
- → Repetirlo con una estructura: se copia campo por campo.

Qué se copia exactamente

Tipo del parámetro	Qué ocurre	Coste
`int`, `double`, `char`, punteros	se copian los bits	uno o dos ciclos
estructura o clase	se invoca el **constructor de copia**	depende de la clase
`std::string`, `std::vector`	copia **profunda** de todo el contenido	proporcional al tamaño
arreglo crudo	**no se copia**: decae a puntero	constante

La última fila es la excepción del lenguaje. Es el tema del bloque 8.

El intercambio que no intercambia

La lógica es correcta. El resultado es nulo.

```
void intercambia_mal(int a, int b) {
    int temporal = a;
    a = b;
    b = temporal;
    // aquí a y b sí están intercambiadas
}                              // ...y aquí mueren las dos

int main() {
    int x = 1, y = 2;
    intercambia_mal(x, y);
    std::cout << x << y;       // imprime 1 2
}
```

¿Entonces qué falla?

Demostración en vivo

03_swap_roto.cpp

```
./build/bin/exp_03_swap_roto
```

→ La función **sí** intercambia: sus copias.

→ Al terminar, el marco se libera y las copias desaparecen.

→ Para modificar los argumentos, la función necesita saber **dónde viven**.

Ventajas del paso por valor

- **Aísla al llamador.** La función no puede alterar sus datos.
- **Sin problemas de vida.** El parámetro es dueño de su valor; no puede quedar colgante.
- **Sin aliasing.** El compilador sabe que no comparte memoria con nada y optimiza mejor.
- **Rápido para tipos pequeños.** Un `int` viaja en un registro; una referencia obliga a leer memoria.

Límites del paso por valor

- **No puede modificar el argumento.** El caso canónico es el intercambio.
- **Cuesta.** Para objetos grandes, la copia se paga en tiempo y memoria.
- **No admite tipos no copiables**, como `std::unique_ptr` o `std::mutex`.
- **Trunca el polimorfismo** (*object slicing*): al pasar un objeto derivado por valor en un parámetro de la clase base, la parte derivada se pierde.

Object slicing

```
struct Figura  { virtual double area() const { return 0; } };
struct Circulo : Figura { double r; double area() const override; };

void dibuja(Figura f);          // MAL: recibe solo la parte 'Figura'
void dibuja(const Figura& f);   // BIEN: conserva el tipo dinámico

Circulo c;
dibuja(c);   // con la primera firma, el área siempre da 0
```

Compila, se ejecuta y da un resultado incorrecto. Sin ningún aviso.

Demostración en vivo

04_costo_de_la_copia.cpp

```
./build/bin/exp_04_costo_de_la_copia
```

→ Una clase instrumentada **anuncia cada copia** que ocurre.

→ Veinte llamadas con un vector de 5 millones de enteros, por valor y por `const&`.

→ La copia se paga aunque la función solo lea los datos.

Cuándo el paso por valor SÍ es lo correcto

Que no se vayan con la idea de que 'por valor es malo'

- **Tipos baratos de copiar.** Si no supera dos o tres punteros (16–24 bytes en 64 bits) y la copia es trivial, por valor.
- **Cuando la función necesita su propia copia de trabajo.** Si de todos modos vas a copiar dentro, pide la copia en el parámetro.
- **Parámetros sumidero.** Cuando la función se queda con el dato: `Usuario(std::string n) : nombre_(std::move(n)) {}`

F.16 — *For "in" parameters, pass cheaply-copied types by value and others by reference to const*

C++ Core Guidelines · Bjarne Stroustrup y Herb Sutter (eds.)

Práctica 1 · Paso por valor

15 minutos · individual o por parejas, con el compilador abierto · ejercicios/practica-1-paso-de-argumentos.md

1. Predecir la salida de un intercambio aritmético por valor.
2. Encontrar y corregir el error en una función que debería duplicar.
3. Elegir la forma de paso para cinco funciones y justificarla.
4. Reto: contar cuántas veces se ejecuta el constructor de copia.

4

Paso de argumentos por referencia

35 minutos

- La referencia es un alias
- El intercambio que sí funciona
- const T&: leer sin copiar
- Referencias colgantes

Qué es una referencia

No es un objeto nuevo. No es un puntero con azúcar sintáctico.

- Una referencia `T&` es **otro nombre para un objeto que ya existe**.

  — **Debe inicializarse** al declararse. No existe una referencia vacía.
  — **No se puede reapuntar.** Queda ligada a ese objeto de por vida.
  — **No puede ser nula.** Siempre hay un objeto detrás.

- La comprobación cabe en una línea: `&referencia == &objeto` es **verdadero**.

El alias, en código

```
int valor = 42;
int& alias = valor;          // 'alias' NO es un objeto nuevo

&valor == &alias             // true: misma dirección, mismo objeto

alias = 7;                   // valor ahora vale 7

int otro = 100;
alias = otro;                // asigna el VALOR de otro a valor;
                             // alias SIGUE siendo alias de valor
```

Escribir en el alias es escribir en el objeto original.

Demostración en vivo

06_referencias_basico.cpp

```
./build/bin/exp_06_referencias_basico
```

- → `&valor == &alias` da verdadero: es el mismo objeto.
- → Mostrar las dos declaraciones comentadas que **no compilan**.
- → Un parámetro `int&` modifica el objeto del llamador.

El mismo cuerpo, dos símbolos de diferencia

La respuesta a la pregunta del bloque 2

No funciona

```
void intercambia_mal(int a, int b)
{
    int temporal = a;
    a = b;
    b = temporal;
}

int x = 1, y = 2;
intercambia_mal(x, y);
// x=1  y=2
```

Funciona

```
void intercambia(int& a, int& b)
{
    int temporal = a;
    a = b;
    b = temporal;
}

int x = 1, y = 2;
intercambia(x, y);
// x=2  y=1
```

El cuerpo es idéntico. Cambian dos símbolos & en la lista de parámetros.

Demostración en vivo

07_swap_por_referencia.cpp

```
./build/bin/exp_07_swap_por_referencia
```

- → La versión con referencias, la versión de C con punteros y `std::swap`.
- → Las tres hacen lo mismo; cambia la sintaxis y las garantías.
- → Con punteros hay que comprobar `nullptr`; con referencias, no.

En el punto de llamada, `f(x)` se ve **exactamente igual** con paso por valor y con paso por referencia.

La declaración de la función es la única fuente de información. Por eso hay que leer las firmas.

Las tres formas de parámetro por referencia

- `const T&` — referencia a const: **entrada**, sin copiar. La más común de C++.
- `T&` — referencia no const: **entrada y salida**. Modifica el argumento.
- `T&&` — referencia a rvalue: **sumidero**, se va a mover.

- `T&&` pertenece a la semántica de movimiento y queda fuera de esta sesión. Basta con saber que existe.

const T&: no copia y no modifica

```
std::size_t longitud(const std::string& texto) {
    // texto[0] = 'X';   // error: cannot assign to variable
    return texto.size(); //        with const-qualified type
}

void agrega_saludo(std::string& texto) {   // sin const: SÍ modifica
    texto = "Hola, " + texto;
}
```

const no es cosmética: es la única documentación que el compilador verifica.

const T& acepta argumentos que T& rechaza

Llamada	`const std::string&`	`std::string&`
`f(nombre)` — un lvalue	sí	sí
`f("literal")` — conversión	sí	**no**
`f(std::string{"t"})` — temporal	sí	**no**
`f(objeto_const)`	sí	**no**

Permitir modificar un temporal que va a morir en la siguiente línea casi siempre es un error. El lenguaje lo prohíbe.

Demostración en vivo

08_referencia_const.cpp

```
./build/bin/exp_08_referencia_const
```

→ `const&` no copia, no modifica y acepta temporales y objetos `const`.

→ Descomentar en vivo las dos llamadas que **no compilan**.

→ Vida extendida de un temporal ligado a `const&`.

Una referencia **no es dueña** del objeto al que apunta.

Si el objeto muere antes que la referencia, usarla es comportamiento indefinido.

Las dos causas de referencia colgante

A · devolver una local

```
int& colgante() {
    int local = 1234;
    return local;      // muere aquí
}

int& r = colgante();
std::cout << r;        // basura

// El compilador SÍ avisa:
// warning: reference to stack
// memory ... returned
```

B · el contenedor crece

```
std::vector<int> v{10, 20, 30};
int& primero = v[0];

for (int i = 0; i < 1000; ++i)
    v.push_back(i);    // realoja

std::cout << primero;  // colgante

// El compilador NO avisa.
// AddressSanitizer sí.
```

Solución práctica: no devuelvas referencias a locales, y guarda el ÍNDICE, no la referencia.

Demostración en vivo

09_referencias_colgantes.cpp

```
ASAN_OPTIONS=detect_stack_use_after_return=1 \
    ./build/bin/exp_09_referencias_colgantes
```

- → Ejecutarlo **primero sin** la variable de entorno: termina sin quejarse.
- → Ejecutarlo **después con** ella: AddressSanitizer aborta y señala la línea.
- → El mismo programa, dos comportamientos. Eso es comportamiento indefinido.

Devolver resultados: dos estilos

F.20 y F.21 de las Core Guidelines

Parámetros de salida

```
void estadisticas(
    const std::vector<double>& d,
    double& minimo,
    double& maximo,
    double& media);

double mn = 0, mx = 0, md = 0;
estadisticas(datos, mn, mx, md);

// ¿cuáles entran y cuáles salen?
// hay que ir a ver la firma
```

Valor de retorno (preferido)

```
struct Estadisticas {
    double minimo, maximo, media;
};

Estadisticas estadisticas(
    const std::vector<double>& d);

auto [mn, mx, md] =
    estadisticas(datos);

// se lee sola
```

El valor de retorno no se puede ignorar por accidente y no obliga a declarar variables vacías antes.

Demostración en vivo

10_salida_por_parametro.cpp

```
./build/bin/exp_10_salida_por_parametro
```

- → Los tres estilos: parámetros de salida, `struct` de retorno y entrada/salida real.
- → `T&` sigue siendo correcto cuando el parámetro **entra con un valor y sale modificado**.
- → Ejemplo: normalizar un vector en el sitio.

5

const, punteros y referencias

20 minutos

- Cuándo usar cada uno
- El puntero también viaja por valor
- Dónde va el const
- Tabla de decisión

Referencia y puntero

	`T&` referencia	`T*` puntero
Puede ser nulo	no	sí (`nullptr`)
Se puede reapuntar	no	sí
Debe inicializarse	sí	no
En la llamada	`f(x)`	`f(&x)`
Al usarlo	x	`*p` `p->`
Aritmética	no	sí (`p + 1`)

No son intercambiables: se diferencian en lo que PERMITEN.

F.60 — *Prefer T* over T& when "no argument" is a valid option*

Traducido: referencia por omisión; puntero cuando la ausencia de dato sea un caso legítimo.

El punto que siempre se confunde

Un puntero también se pasa por valor

```
void f(int* p) {
    *p = 10;            // SÍ afecta al objeto del llamador
    p = nullptr;        // NO afecta al puntero del llamador
}
```

La función recibe una COPIA de la dirección. Para cambiar el puntero del llamador hace falta int*& o int**.

Dónde va el const

Declaración	Lectura (de derecha a izquierda)	`*p = 5`	`p = &otro`
`int* p`	puntero a int	sí	sí
`const int* p`	puntero a int **constante**	**no**	sí
`int* const p`	puntero **constante** a int	sí	**no**
`const int* const p`	puntero constante a int constante	**no**	**no**

const int* e int const* son el mismo tipo. Con referencias no hay ambigüedad: solo const T& y T&.

Tabla de decisión

La diapositiva que conviene fotografiar

Intención	Parámetro	Regla
Entrada, tipo barato de copiar	T	F.16
Entrada, tipo caro de copiar	`const T&`	F.16
Entrada y salida	T&	F.17
Solo salida	valor de retorno	F.20
Varias salidas	`struct` de retorno	F.21
Puede no haber argumento	`T*` o `std::optional`	F.60
Secuencia contigua	`std::span<T>`	F.24, I.13

Demostración en vivo

11 y 12 · punteros, referencias y const

```
./build/bin/exp_11_punteros_vs_referencias
./build/bin/exp_12_const_y_punteros
```

- → Argumento obligatorio frente a argumento opcional.
- → Aritmética de punteros para recorrer.
- → Las cuatro combinaciones de `const`, con los errores comentados.

Práctica 2 · Referencias, punteros y const

15 minutos · por parejas · ejercicios/practica-2-referencias-y-const.md

1. Escribir cuatro firmas de función a partir de su intención.
2. Detectar dos errores de vida y decir cuál atrapa el compilador.
3. Completar la tabla de lectura de const.
4. Reto: la misma función con punteros y con referencias.

7

Arreglos unidimensionales

25 minutos

- Bloque contiguo, tamaño fijo
- Índices y direcciones
- Sin comprobación de límites
- Algoritmos típicos

Un arreglo es un **bloque contiguo de N objetos del mismo tipo**, con N fijo y conocido en tiempo de compilación.

Todo lo demás se deduce de esas dos propiedades: contigüidad y tamaño fijo.

El tamaño debe ser constante

```
int v[5];              // bien: 5 es una expresión constante

const int N = 10;
int w[N];              // bien: N es constante

int n;
std::cin >> n;
int malo[n];           // NO es C++ estándar (esto es un VLA de C99)

std::vector<int> bien(n);   // ESTA es la respuesta correcta
```

[dcl.array]/1 — el tamaño debe ser una expresión constante convertida de tipo std::size_t, mayor que cero.

Formas de inicializar

Declaración	Resultado
`int a[5] = {10,20,30,40,50};`	los cinco valores dados
`int b[5] = {1,2};`	`1 2 0 0 0` — lo que falta se pone a cero
`int c[5] = {};`	los cinco a cero
`int d[] = {7,8,9};`	tamaño deducido: 3
`int e[3];`	valores **indeterminados**: leerlos es comportamiento indefinido

Desde C++11 se puede omitir el =: int a[5]{10,20,30,40,50};

Índices y direcciones

Por qué los índices empiezan en cero

```
v[i]  ==  *(v + i)              // por definición del operador []

&v[i] ==  dirección_base + i * sizeof(T)
```

El índice es un DESPLAZAMIENTO desde el inicio, no una posición ordinal. El primer elemento está a desplazamiento cero.

Tamaño y recorrido

```
sizeof(a)                   // bytes que ocupa TODO el arreglo
sizeof(a) / sizeof(a[0])    // número de elementos, forma clásica
std::size(a)                // número de elementos, C++17, <iterator>

for (std::size_t i = 0; i < std::size(a); ++i)  { ... a[i] ... }

for (int valor : a)         { ... }        // C++11, sin índices
for (int& valor : a)        { valor *= 2; } // con & para modificar
```

std::size es preferible: falla al compilar si se aplica a un puntero, que es justo el error que la forma clásica no detecta.

Demostración en vivo

13_arreglo_basico.cpp

```
./build/bin/exp_13_arreglo_basico
```

- → Las cinco formas de inicializar, con su resultado real.
- → Las direcciones de los elementos, separadas exactamente por `sizeof(T)`.
- → Recorrido clásico, por rango y por rango con referencia.

Un arreglo **no comprueba sus límites**. `v[10]` en un arreglo de cinco elementos compila y se ejecuta.

Lee o escribe memoria que no le pertenece. El estándar lo llama comportamiento indefinido.

Qué sí avisa

Herramienta	Qué detecta	Cuándo
`-Warray-bounds`	índice **constante** fuera de rango	compilación
`array::at()` / `vector::at()`	cualquier índice fuera de rango	ejecución, lanza `std::out_of_range`
AddressSanitizer	accesos fuera de rango reales	ejecución
UndefinedBehaviorSanitizer	índices fuera de rango, entre otras cosas	ejecución

El compilador solo puede avisar de lo que puede calcular. Si el índice se conoce hasta la ejecución, no hay aviso.

El error de un elemento

off by one

```
for (int i = 0; i <= 5; ++i)   // MAL: recorre SEIS posiciones
for (int i = 0; i <  5; ++i)   // BIEN
```

El índice válido máximo es N-1. La condición correcta usa <, nunca <=.

Demostración en vivo

14_fuera_de_rango.cpp

```
./build/bin/exp_14_fuera_de_rango
ASAN_OPTIONS=detect_stack_use_after_return=1 ./build/bin/exp_14_fuera_de_rango
```

→ Ejecutarlo **dos veces**: sin sanitizador, termina; con él, aborta.

→ `std::array::at()` lanza `std::out_of_range` y se puede atrapar.

→ Los centinelas alrededor del arreglo muestran qué se corrompe.

Lo que un arreglo NO puede hacer

- **No se puede asignar**: `a = b;` no compila.
- **No se puede copiar** al inicializar: `int b[5] = a;` no compila.
- **No se puede devolver** desde una función.
- **No se puede comparar**: `a == b` compara direcciones, no contenidos.
- **No conoce su tamaño** una vez pasado a una función.

- `std::array<T,N>` sí puede hacer todo eso, con el mismo rendimiento y sin sobrecosto de memoria.

Demostración en vivo

15_arreglo_algoritmos.cpp

```
./build/bin/exp_15_arreglo_algoritmos
```

- → Media, máximo, búsqueda lineal y ordenamiento por burbuja.
- → Todos comparten el mismo esqueleto de recorrido.
- → Al final, los equivalentes de `<algorithm>`: `sort`, `find`, `max_element`.

8

Arreglos como parámetros: el decaimiento

20 minutos

- La bisagra entre los dos temas de la sesión

¿Un arreglo se pasa por valor o por referencia?

Ni una cosa ni la otra.

Las dos reglas de la norma

ISO/IEC 14882 — decaimiento, array-to-pointer decay

```
[conv.array]/1
  "Una expresión E de tipo 'arreglo de N T' puede convertirse
   a un prvalue de tipo 'puntero a T'."

[dcl.fct]/5
  "Después de determinar el tipo de cada parámetro, todo parámetro
   de tipo 'arreglo de T' se ajusta para ser 'puntero a T'."
```

La primera describe la conversión en el punto de llamada; la segunda, el ajuste del parámetro.

Las tres declaraciones son la MISMA función

```
void f(int v[10]);    // el 10 se ignora por completo
void f(int v[]);
void f(int* v);       // esta es la firma real de las tres
```

Declarar las tres en el mismo programa da error: redefinition of 'f'.

sizeof deja de servir

En main

```
int datos[10];

sizeof(datos);      // 40

std::size(datos);   // 10

for (int x : datos) // compila
```

Dentro de la función

```
void f(int v[10]) {

    sizeof(v);      // 8

    // std::size(v) NO compila

    // for (int x : v) NO compila
}
```

Calcular sizeof(v)/sizeof(v[0]) dentro de la función es uno de los errores más frecuentes del lenguaje.

Como solo viaja la dirección, un arreglo **nunca se copia** al pasarlo, y la función **sí puede modificar** los elementos del llamador.

Es la única excepción a la regla 'por valor significa copia'.

Demostración en vivo

16_decaimiento.cpp

```
./build/bin/exp_16_decaimiento
```

- `sizeof` vale 40 fuera de la función y 8 dentro.
- La función modifica el arreglo del llamador sin haberlo copiado.
- El decaimiento también ocurre fuera de las llamadas: `int* p = datos;`

Solución 1 · conservar el tamaño

Tamaño fijo

```
void f(int (&v)[10]);

// los paréntesis son
// obligatorios:
// int& v[10] sería un
// arreglo de referencias,
// que no existe en C++
```

Tamaño deducido

```
template <std::size_t N>
void g(const int (&v)[N]);

// N lo deduce el compilador
// en cada llamada.
// Dentro funcionan sizeof,
// std::size y el for por rango
```

La referencia a arreglo conserva el tamaño porque el tamaño forma parte del tipo.

Solución 2 · std::span (C++20)

dirección + tamaño, en un objeto de 16 bytes

```
double media(std::span<const double> datos);

double crudo[4]          = {1, 2, 3, 4};
std::array<double,3> a   = {10, 20, 30};
std::vector<double> v    = {5, 5, 5, 5, 5};

media(crudo);    // las tres llamadas usan
media(a);        // la MISMA firma
media(v);
```

I.13 — Do not pass an array as a single pointer. · F.24 — Use a span<T> to designate a half-open sequence.

Las cuatro formas de recibir una secuencia

Forma	Conserva tamaño	Cualquier tamaño	Acepta vector	Estándar
`T* v, std::size_t n`	manual	sí	con `.data()`	C++98
`T (&v)[N]`	sí	no	no	C++98
`template<N> T (&v)[N]`	sí	sí	no	C++98
`std::span<T>`	sí	sí	**sí**	C++20
`const std::vector<T>&`	sí	sí	sí	C++98

Demostración en vivo

17 y 18 · conservar el tamaño

```
./build/bin/exp_17_arreglo_por_referencia
./build/bin/exp_18_span
```

- → Plantilla con `N` deducido: una sola función para todos los tamaños.
- → `std::span` con arreglo crudo, `std::array` y `std::vector`.
- → Subvistas sin copiar: `first`, `last`, `subspan`.

9

Arreglos multidimensionales

25 minutos

- Un arreglo de arreglos
- Orden por filas
- Pasar matrices
- Por qué el orden de los bucles cambia el rendimiento

int m[3][4] es un arreglo de arreglos

[dcl.array]/9 — cuando varias especificaciones 'array of' son adyacentes se crea un tipo multidimensional

```
int m[3][4];       // 3 elementos, y cada uno es un arreglo de 4 int

m          es de tipo   int[3][4]
m[i]       es de tipo   int[4]     <- UNA FILA ENTERA, no un elemento
m[i][j]    es de tipo   int
```

De esta frase salen todas las reglas del tema.

Orden por filas (row-major)

```
int m[3][4] en memoria, un único bloque contiguo:

[ m00 m01 m02 m03 | m10 m11 m12 m13 | m20 m21 m22 m23 ]
  ---- fila 0 ----   ---- fila 1 ----   ---- fila 2 ----
índice lineal:
    0   1   2   3      4   5   6   7      8   9  10  11

     &m[i][j]  ==  base + (i * COLUMNAS + j) * sizeof(T)
```

Fortran, MATLAB, R y Julia usan el orden inverso, columna por columna. Es una causa clásica de errores al portar código numérico.

Demostración en vivo

19_matriz_basica.cpp

```
./build/bin/exp_19_matriz_basica
```

→ Los desplazamientos en bytes **confirman la fórmula** `i*C + j`.

→ `sizeof(m)`, `sizeof(m[0])` y `sizeof(m[0][0])`: matriz, fila y elemento.

→ `m[1][2]` y `*(m[1]+2)` son la misma expresión.

for basado en rango sobre una matriz

```
for (const auto& fila : m) {      // fila es 'const int (&)[4]'
    for (int valor : fila) {
        ...
    }
}

// for (auto fila : m)   <- MAL: intentaría copiar un arreglo;
//                          fila decae a puntero y el bucle
//                          interno deja de compilar
```

La variable externa DEBE ser referencia a fila.

Pasar una matriz a una función

```
void f(int m[][4], int filas);    // válido: solo la 1ª dimensión se omite
void f(int m[3][4], int filas);   // el 3 se ignora
void f(int (*m)[4], int filas);   // la firma REAL de las dos anteriores

void f(int** m, ...);             // ERROR conceptual: no es una matriz
```

int[3][4] decae a 'puntero a arreglo de 4 int', es decir int (*)[4]. NO decae a int**.

Por qué int** no funciona

Disposiciones de memoria incompatibles

```
int m[3][4]         un único bloque de 12 enteros
                    [ ][ ][ ][ ][ ][ ][ ][ ][ ][ ][ ]

int**               un arreglo de DIRECCIONES, cada una
                    apuntando a una fila suelta
                    [ *]--> [ ][ ][ ][ ]
                    [ *]--> [ ][ ][ ][ ]
                    [ *]--> [ ][ ][ ][ ]
```

error: cannot initialize a parameter of type 'int **' with an lvalue of type 'int[3][4]'

Demostración en vivo

20 y 21 · matrices como parámetros y operaciones

```
./build/bin/exp_20_matriz_parametros
./build/bin/exp_21_matriz_operaciones
```

- → Las cuatro formas de recibir una matriz, incluida la vista aplanada.
- → Suma, transpuesta y producto con plantillas.
- → Al llevar las dimensiones en el tipo, **el compilador verifica** la compatibilidad.

Tres dimensiones y la fórmula general

```
T v[D0][D1][D2]

  v[i][j][k]  está en  base + ( i*D1*D2 + j*D2 + k ) * sizeof(T)


Cada índice se multiplica por el producto de todas las
dimensiones que quedan a su derecha.
```

Es lo que dice el ejemplo de [dcl.array]/9 de la norma.

El orden de los bucles cambia el rendimiento

Matriz de 2048 × 2048 int (16 MiB), clang -O2, Apple Silicon

Recorrido	Código	Tiempo
Por filas	`for (i) for (j) suma += m[i][j];`	**0.88 ms**
Por columnas	`for (j) for (i) suma += m[i][j];`	**25.84 ms**

Mismo resultado numérico. Casi 30 veces más lento.

En C y C++, el **último índice** debe ser el del bucle **más interno**.

Cambiar el orden de dos bucles anidados es la optimización más barata que existe en cómputo con matrices.

Demostración en vivo

23_orden_de_recorrido.cpp

```
clang++ -std=c++20 -O2 exposicion/src/23_orden_de_recorrido.cpp -o /tmp/recorrido
/tmp/recorrido
```

- → Compilarlo **aparte con -O2**: los sanitizadores distorsionan la medición.
- → Las dos sumas dan el mismo número.
- → La diferencia de tiempo es de un orden de magnitud.

10

Qué usar en lugar del arreglo crudo

10 minutos

- std::array, std::vector, std::span
- Matrices modernas
- Cierre

Las tres herramientas

Herramienta	Cuándo	Qué aporta
`std::array<T,N>`	tamaño fijo, conocido al compilar	no decae, conoce su tamaño, se copia y **se devuelve**
`std::vector<T>`	tamaño decidido al ejecutar	crece, se redimensiona, libera sola su memoria
`std::span<T>`	**parámetros** de función	dirección + tamaño; una firma para los tres

std::array no tiene ningún sobrecosto: sizeof(std::array<int,4>) == sizeof(int[4]) == 16.

Matrices en C++ moderno

Opción	Contigua	Dimensiones	Nota
`std::array<std::array<T,C>,F>`	sí	fijas	equivalente de `T m[F][C]`
`std::vector<T>` aplanado + `i*C+j`	sí	en ejecución	lo que usan las bibliotecas numéricas
`std::vector<std::vector<T>>`	**no**	en ejecución	cómodo pero **lento**: cada fila es un bloque suelto
`std::mdspan` (C++23)	sí	en ejecución	vista multidimensional sobre un bloque lineal

Una matriz que reúne todo lo visto hoy

```
class Matriz {
public:
    Matriz(std::size_t filas, std::size_t columnas)
        : filas_(filas), columnas_(columnas),
          datos_(filas * columnas, 0.0) {}

    double& operator()(std::size_t i, std::size_t j)
        { return datos_[i * columnas_ + j]; }

    double operator()(std::size_t i, std::size_t j) const
        { return datos_[i * columnas_ + j]; }

private:
    std::size_t filas_, columnas_;
    std::vector<double> datos_;      // contiguo, como el arreglo crudo
};
```

Memoria contigua · la fórmula i*C+j · dimensiones que viajan con el objeto · const correcto en las dos sobrecargas.

Criterio final

Situación	Elección
Tamaño fijo y conocido al compilar	`std::array<T,N>`
Tamaño decidido al ejecutar	`std::vector<T>`
Parámetro que recibe una secuencia	`std::span<T>`
Parámetro de entrada, objeto grande	`const T&`
Interoperar con C, o sistema embebido	arreglo crudo

Los tres mensajes de la sesión

- **Elegir cómo se pasa un argumento es elegir qué se copia.** Por valor copia el objeto; por referencia no copia nada y crea un alias.

- **Un arreglo nunca se copia al pasarlo: decae a puntero** y pierde su tamaño. Por eso hay que pasar el tamaño aparte, o usar `std::span`.

- **La memoria es contigua y por filas.** De ahí salen el índice base cero, la fórmula `i*C + j` y una diferencia de 30 veces por cambiar el orden de dos bucles.

Para seguir

Bibliografía y material de apoyo

- **C++ Core Guidelines** — isocpp.github.io/CppCoreGuidelines · reglas F.15 a F.24, F.60, I.11, I.13
- **Deitel & Deitel**, *Cómo programar en C++*, 9.ª ed. — caps. 6, 7 y 8
- **Stroustrup**, *The C++ Programming Language*, 4.ª ed. — caps. 7 y 12
- **Meyers**, *Effective C++*, 3.ª ed. — ítem 20
- **eel.is/c++draft** — el texto de la norma, con enlace por párrafo
- **godbolt.org** — Compiler Explorer, para ver el código generado

- Base de conocimiento, 24 ejemplos y tres prácticas: `Repositorios/C-CPP/exposicion/`

Gracias. ¿Preguntas?

Alberto López · Programación Avanzada · UACBI, UAN